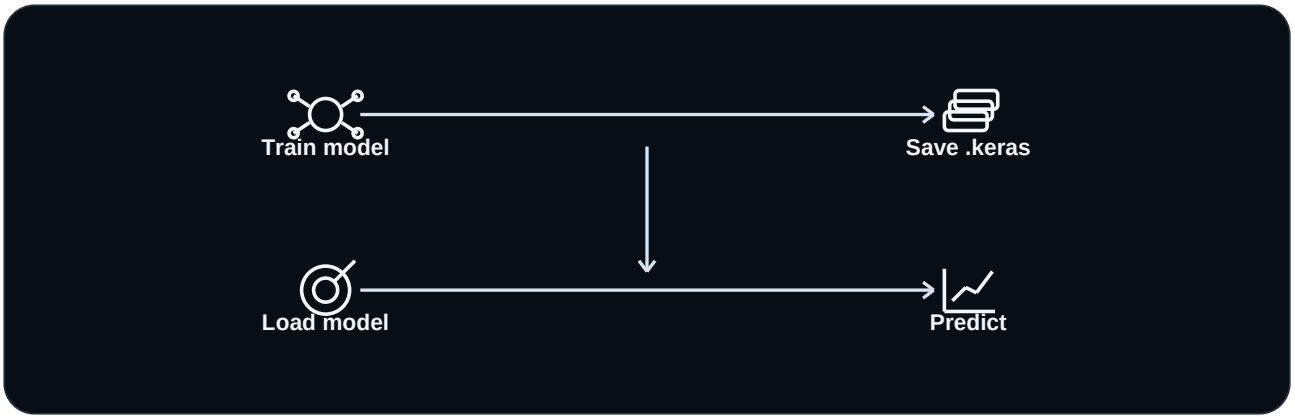
TENSORFLOW

A visual learning guide | Days 01-12



A practical, visual record of the concepts, code patterns and environment setup covered so far.

THE PROJECT

football-tensorflow-player-predictor
Windows CMD | Python 3.13.14 | TensorFlow 2.21.0 | Virtual environment: **venv**

How to use this guide

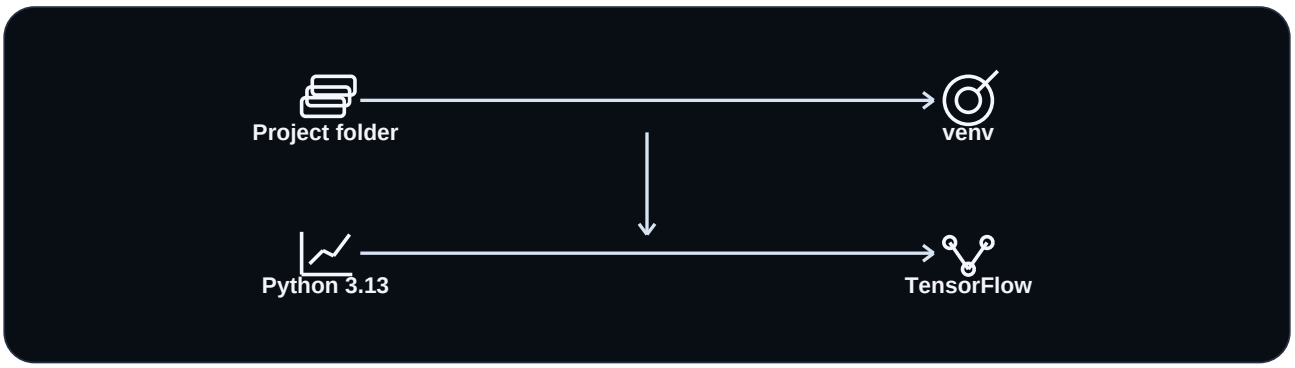
Each day follows the same learning sequence: concept, visual flow, formula or intuition, code pattern, and interpretation. Use the final template to record every new day before committing it to GitHub.

Inside this guide

Environment and venv setup | Days 01-12 | Tensor shapes | Keras method reference | Troubleshooting | Reusable learning log

01 | Environment setup

A virtual environment isolates a project's Python interpreter and packages. It prevents dependencies for one project from interfering with another.



Recommended structure

```
football-tensorflow-player-predictor/
|-- data/
|-- models/
|-- notebooks/
|-- src/
|   |-- 01_tensors.py
|   |-- 02_tensors_operations.py
|   |-- ... day files ...
|-- venv/                # local only; do not commit
|-- requirements.txt
|-- .gitignore
|-- README.md
```

Create and activate the environment

```
# Run from the project root
py -3.13 -m venv venv
venv\Scripts\activate

python -m pip install --upgrade pip
python -m pip install tensorflow
```

Verify the active interpreter

```
where python
python --version
python -m pip show tensorflow
python -c "import tensorflow as tf; print(tf.__version__)"
```

IMPORTANT: THE WINDOWS INTERPRETER ISSUE

Your terminal displayed (venv), but where python still returned WindowsApps / a global Python. The working interpreter was **venv\Scripts\python.exe** (Python 3.13.14). If activation is unreliable, bypass it and run **venv\Scripts\python.exe src\your_script.py**.

Confirmed setup and GPU note

TensorFlow 2.21.0 was installed under venv\Lib\site-packages. The standard TensorFlow package reported that GPU support is unavailable on native Windows; this is an environment limitation, not a model error. These exercises ran on CPU. For GPU workflows, use a supported setup such as WSL2.

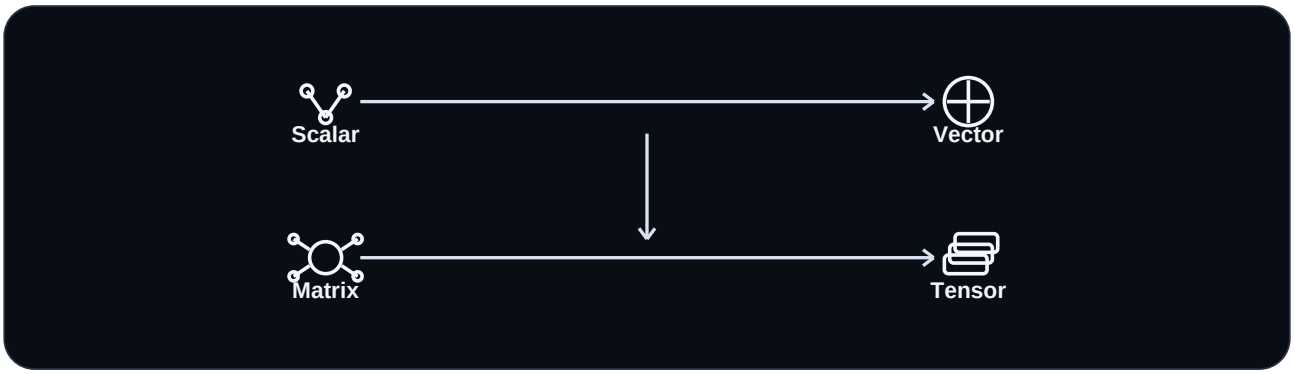
Useful commands

```
# From project root
venv\Scripts\activate
deactivate
where python
py -0p
python --version
python -m pip show tensorflow

# From src: run with the correct interpreter
..\venv\Scripts\python.exe 12_save_load.py
```

01 | Tensor basics

A tensor is TensorFlow's main data container. It can represent a scalar, vector, matrix, or higher-dimensional data.



Core idea

A tensor has a value, a shape, and a dtype. Rank is the number of dimensions; shape gives the size of each dimension.

Code pattern

```
import tensorflow as tf
values = tf.constant([10, 20, 30, 40])
print(values)
print(values.shape)
print(values.dtype)
```

What the result means

Creates a rank-1 tensor with shape (4,). Integer values usually produce an integer dtype such as int32.

TAKEAWAY

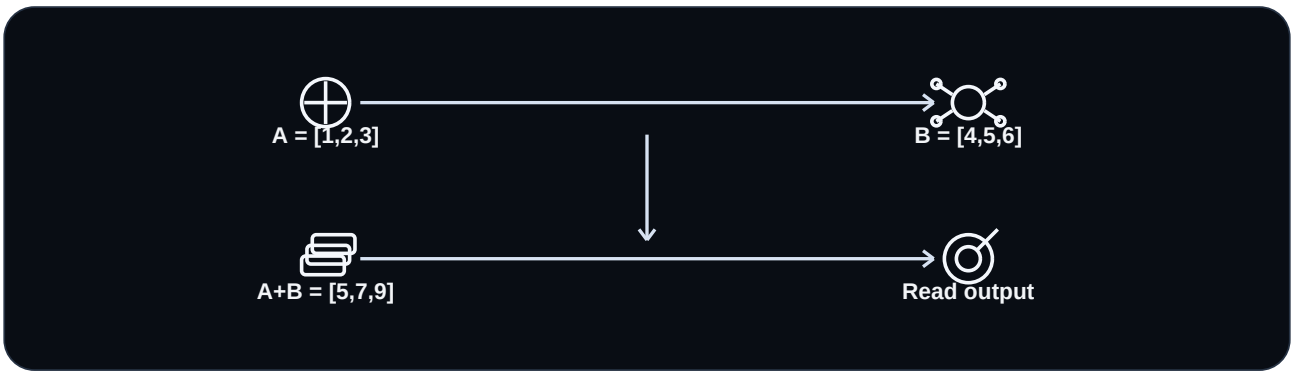
tf.constant(...) creates a tensor. shape describes dimensions. dtype describes the element type.

Remember

- tf.constant(...) creates a tensor.
- shape describes dimensions.
- dtype describes the element type.

02 | Tensor operations

TensorFlow performs mathematical operations directly on tensors. Element-wise operations act on corresponding positions.



Core idea

For same-shaped vectors, addition and multiplication happen position by position.

Code pattern

```
a = tf.constant([1, 2, 3])
b = tf.constant([4, 5, 6])
print(a + b)
print(a * b)
```

What the result means

Addition gives [5, 7, 9]. Multiplication gives [4, 10, 18]. For matrix multiplication, use tf.matmul(...), not *.

TAKEAWAY

+ is element-wise addition. * is element-wise multiplication. Shapes must be compatible; broadcasting is also supported.

Remember

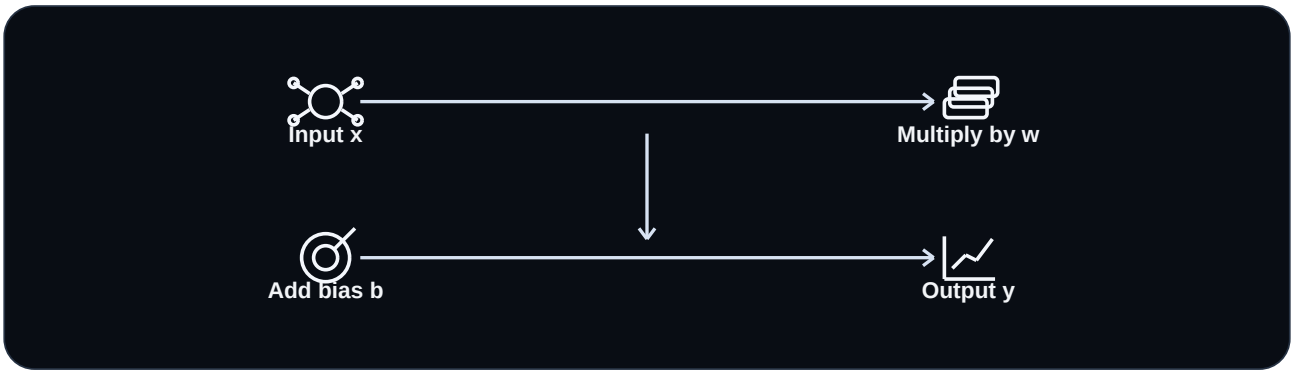
- + is element-wise addition.

- * is element-wise multiplication.

- Shapes must be compatible; broadcasting is also supported.

03 | A single neuron

A basic neuron multiplies an input by a weight and adds a bias.



Core idea

The linear neuron equation is y = w*x + b. The weight controls the input's influence; bias shifts the output.

Code pattern

```
x = tf.constant(2.0)
w = tf.Variable(3.0)
b = tf.Variable(1.0)
y = w * x + b
print(y)
```

What the result means

With x=2, w=3 and b=1, output is 7. tf.Variable creates a value that can be updated during training.

TAKEAWAY

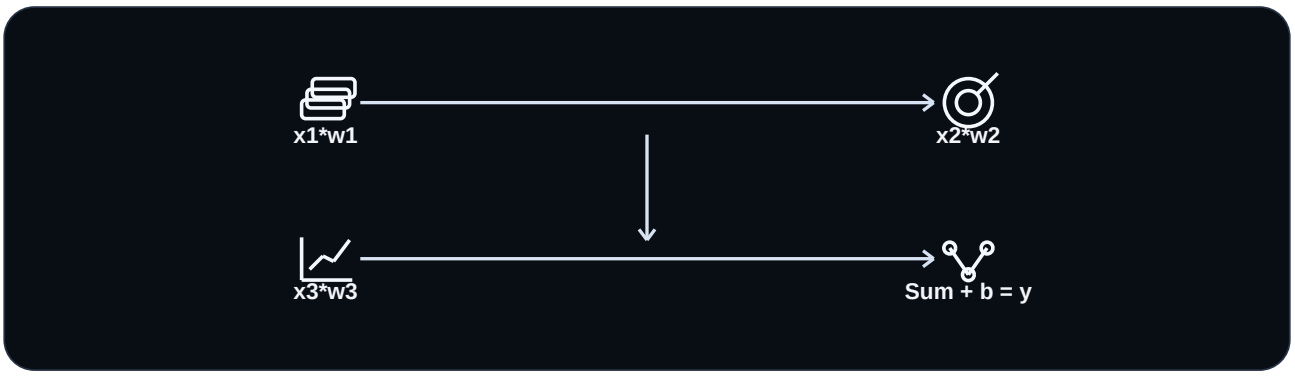
Weights and biases are trainable parameters. Constants are not trainable variables. A neuron first computes a weighted sum plus bias.

Remember

- Weights and biases are trainable parameters.
- Constants are not trainable variables.
- A neuron first computes a weighted sum plus bias.

04 | Multiple inputs

A neuron can receive several features. Each input has its own weight; weighted inputs are summed before adding the bias.



Core idea

For n inputs: y = sum(w_i*x_i) + b.

Code pattern

```
inputs = tf.constant([2.0, 3.0, 4.0])
weights = tf.constant([0.5, 1.0, 2.0])
bias = tf.constant(1.0)
weighted_sum = tf.reduce_sum(inputs * weights)
output = weighted_sum + bias
print(output)
```

What the result means

Products are [1, 3, 8]. Their sum is 12; adding bias 1 gives 13.

TAKEAWAY

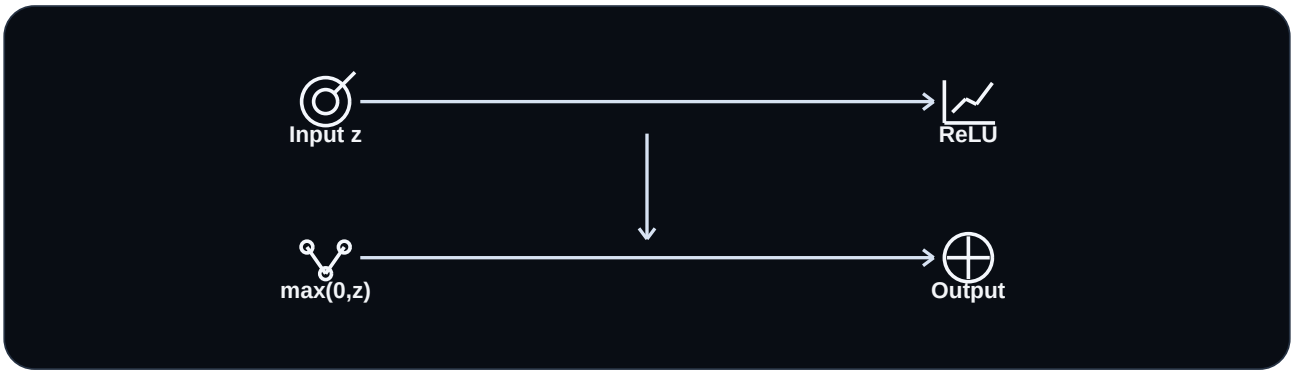
inputs * weights multiplies matching positions. tf.reduce_sum(...) adds tensor values. Bias is added after the weighted sum.

Remember

- inputs * weights multiplies matching positions.
- tf.reduce_sum(...) adds tensor values.
- Bias is added after the weighted sum.

05 | ReLU activation

An activation function transforms a neuron's linear output. ReLU replaces negative values with zero.



Core idea

ReLU(z) = max(0, z). It introduces non-linearity so a network can learn more than a simple linear relationship.

Code pattern

```
values = tf.constant([-3.0, -1.0, 0.0, 2.0, 5.0])
activated = tf.nn.relu(values)
print(activated)
```

What the result means

Output is [0, 0, 0, 2, 5]. Negative values become zero; non-negative values remain unchanged.

TAKEAWAY

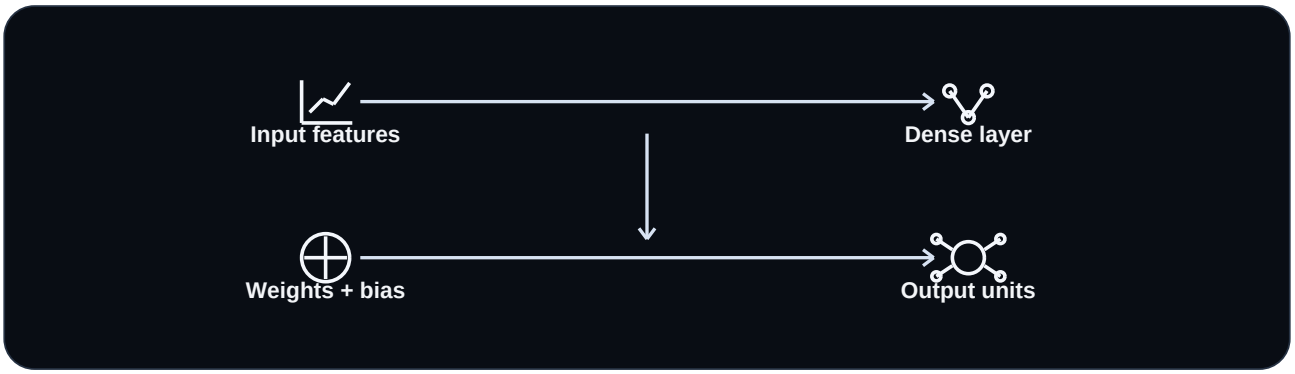
tf.nn.relu(...) applies ReLU. ReLU is common in hidden layers. Output-layer activation depends on the task.

Remember

- tf.nn.relu(...) applies ReLU.
- ReLU is common in hidden layers.
- Output-layer activation depends on the task.

06 | Keras Dense layer

A Dense layer is a fully connected, trainable layer. Every output unit receives all input features.



Core idea

For a layer: output = activation(XW + b). Dense layers manage their weights and biases for you.

Code pattern

```
layer = tf.keras.layers.Dense(4)
x = tf.constant([[1.0, 2.0, 3.0]])
output = layer(x)
print(output)
print("Weights:", layer.weights)
```

What the result means

Dense(4) creates four output units. With three input features, the kernel shape is (3, 4) and bias shape is (4,). Initial values vary.

TAKEAWAY

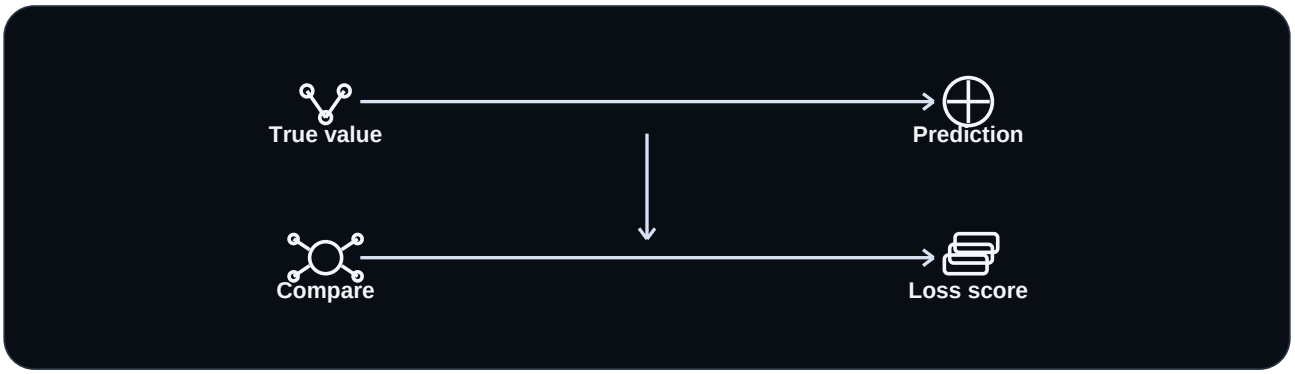
Dense(4) means four output units. Input width is inferred on first call. Default activation is linear.

Remember

- Dense(4) means four output units.
- Input width is inferred on first call.
- Default activation is linear.

07 | Loss function

A loss function measures the difference between a prediction and its true target. Training tries to minimize loss.



Core idea

Squared error for one example is (y_true - y_pred)^2. Mean squared error (MSE) averages squared errors over examples.

Code pattern

```
y_true = tf.constant(10.0)
y_pred = tf.constant(6.0)
loss = tf.square(y_true - y_pred)
print(loss)
```

What the result means

Difference is 4; squared error is 16. A smaller loss means a closer prediction under this loss definition.

TAKEAWAY

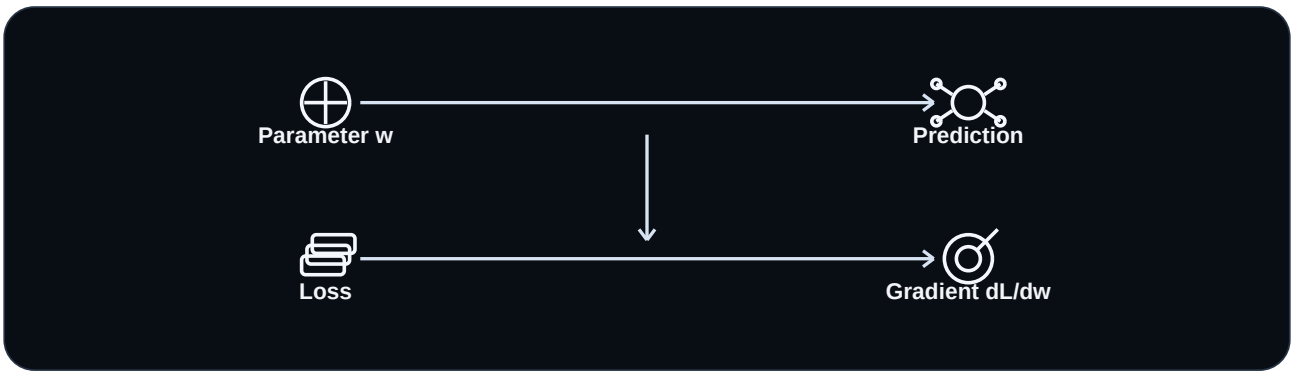
tf.square(...) squares values. MSE is the mean of squared differences. Squaring penalizes larger errors more strongly.

Remember

- tf.square(...) squares values.
- MSE is the mean of squared differences.
- Squaring penalizes larger errors more strongly.

08 | Gradients and GradientTape

A gradient measures how loss changes when a trainable parameter changes. GradientTape records operations so TensorFlow can differentiate them.



Core idea

For x=2, target=10 and w=3, prediction is 6 and loss is 16. The derivative of loss with respect to w is -16.

Code pattern

```
x = tf.constant(2.0)
y_true = tf.constant(10.0)
w = tf.Variable(3.0)
with tf.GradientTape() as tape:
    y_pred = w * x
    loss = tf.square(y_true - y_pred)
gradient = tape.gradient(loss, w)
print("Loss:", loss.numpy())
print("Gradient:", gradient.numpy())
```

What the result means

Loss is 16 and gradient is -16. The negative sign indicates increasing w would locally reduce the loss.

TAKEAWAY

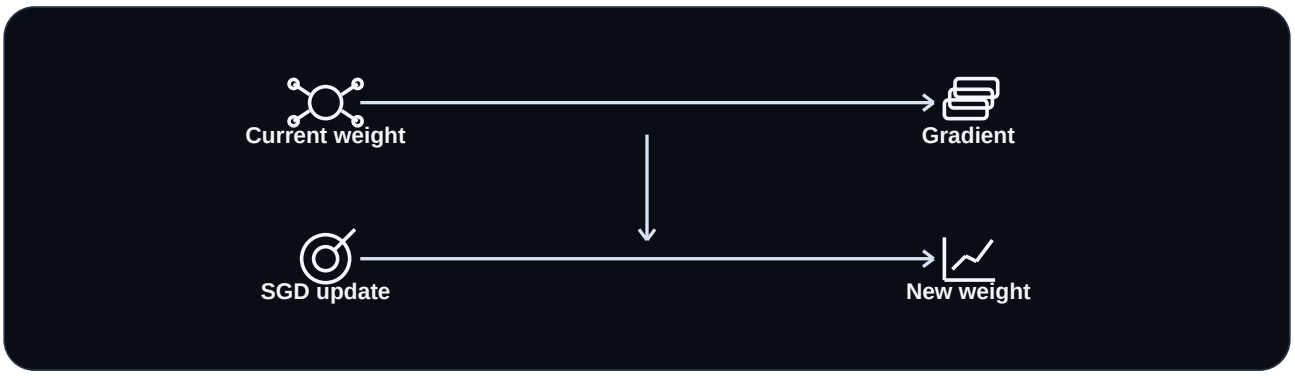
Operations inside GradientTape are recorded. tape.gradient(loss, w) calculates d(loss)/dw. Gradients guide parameter updates.

Remember

- Operations inside GradientTape are recorded.
- tape.gradient(loss, w) calculates d(loss)/dw.
- Gradients guide parameter updates.

09 | Optimizer and SGD

An optimizer uses gradients to update model parameters. Stochastic Gradient Descent moves parameters in a direction that reduces loss.



Core idea

Update rule: w = w - learning_rate * gradient. With w=3, gradient=-16 and learning rate=0.1, the new weight is 4.6.

Code pattern

```
optimizer = tf.keras.optimizers.SGD(learning_rate=0.1)
optimizer.apply_gradients([(gradient, w)])
print("Updated weight:", w.numpy())
```

What the result means

Updated weight is 4.6. For x=2, prediction becomes 9.2 and squared error falls to 0.64.

TAKEAWAY

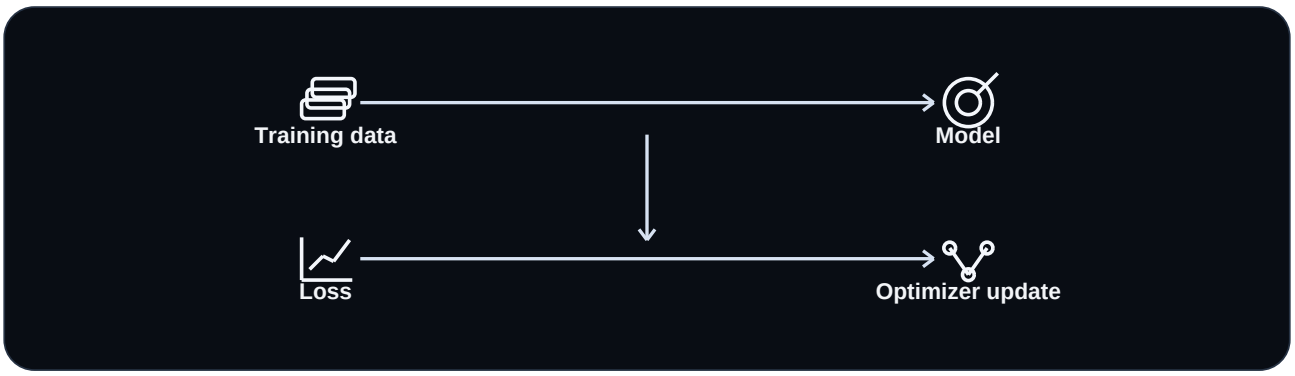
Learning rate controls step size. Too large a rate can overshoot; too small can be slow. apply_gradients receives (gradient, variable) pairs.

Remember

- Learning rate controls step size.
- Too large a rate can overshoot; too small can be slow.
- apply_gradients receives (gradient, variable) pairs.

10 | Training a model

Training repeats prediction, loss calculation, gradient calculation and parameter updates. Keras automates this with model.fit().



Core idea

Your data follows y=2*x. Inputs must be batched: four examples with one feature each have shape (4, 1).

Code pattern

```
x = tf.constant([[1.0], [2.0], [3.0], [4.0]])
y = tf.constant([2.0, 4.0, 6.0, 8.0])
model = tf.keras.Sequential([
    tf.keras.layers.Dense(1)
])
model.compile(
    optimizer=tf.keras.optimizers.SGD(learning_rate=0.1),
    loss="mse"
)
history = model.fit(x, y, epochs=100)
```

What the result means

Your run learned a weight near 2 and bias near 0; prediction for input 5 was about 9.975. Exact values can vary.

TAKEAWAY

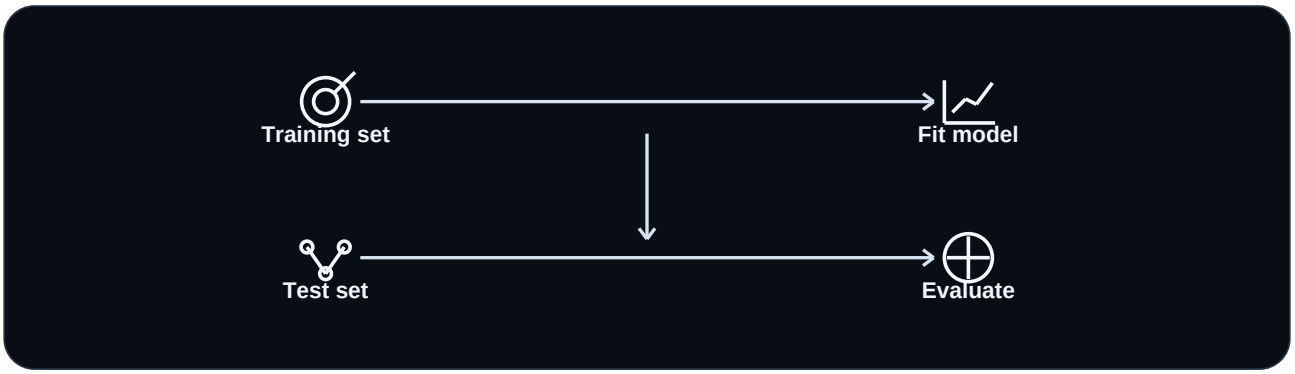
Sequential stacks layers in order. compile(...) configures optimizer and loss. fit(...) trains for the requested epochs. history stores training metrics.

Remember

- Sequential stacks layers in order.
- compile(...) configures optimizer and loss.
- fit(...) trains for the requested epochs.
- history stores training metrics.

11 | Evaluation and generalization

Evaluation measures performance on data not used to fit the model. This helps estimate how well the model generalizes.



Core idea

You trained with x=[1,2,3,4] and tested with x=[5,6,7], targets [10,12,14]. The test set was held out from training.

Code pattern

```
x_train = tf.constant([[1.0], [2.0], [3.0], [4.0]])
y_train = tf.constant([2.0, 4.0, 6.0, 8.0])
x_test = tf.constant([[5.0], [6.0], [7.0]])
y_test = tf.constant([10.0, 12.0, 14.0])
model.fit(x_train, y_train, epochs=100, verbose=0)
test_loss = model.evaluate(x_test, y_test, verbose=0)
predictions = model.predict(x_test, verbose=0)
print("Test loss:", test_loss)
print("Predictions:", predictions.flatten())
print("Actual:", y_test.numpy())
```

What the result means

Recorded test loss was about 0.000262; predictions were close to [10, 12, 14]. fit() updates weights; evaluate() measures loss; predict() returns outputs.

TAKEAWAY

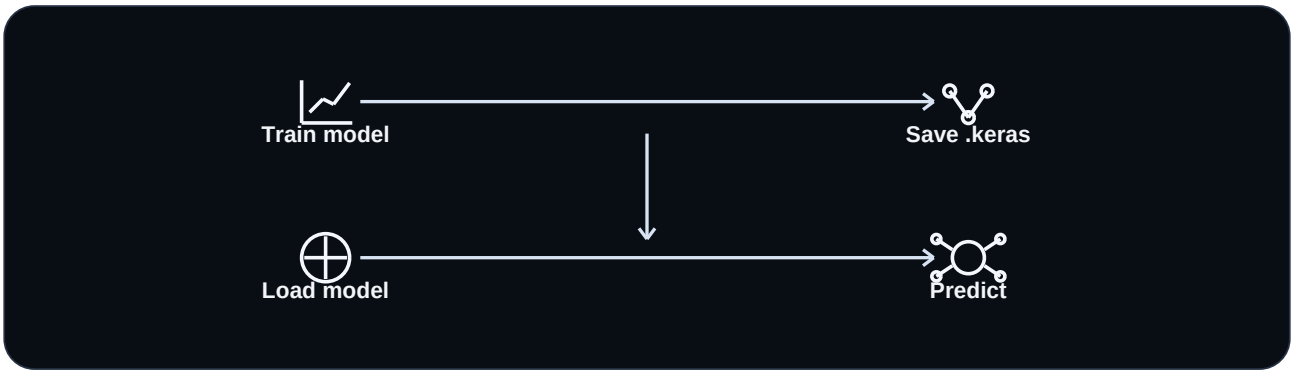
Training data teaches the model. Test data estimates performance on unseen examples. Overfitting means fitting training data well but performing poorly on unseen data.

Remember

- Training data teaches the model.
- Test data estimates performance on unseen examples.
- Overfitting means fitting training data well but performing poorly on unseen data.

12 | Save and load a model

Saving preserves a trained model for later use. Loading restores it without repeating training.



Core idea

You saved to ../models/football_model.keras from src. The .keras format is Keras's native model format.

Code pattern

```
model.save("../models/football_model.keras")
loaded_model = tf.keras.models.load_model(
    "../models/football_model.keras"
)
prediction = loaded_model.predict(
    tf.constant([[5.0]]), verbose=0
)
print("Prediction for 5:", prediction[0][0])
```

What the result means

Your model file was 14,806 bytes. The loaded model predicted 9.982252 for input 5, without training the loaded model again.

TAKEAWAY

model.save(...) writes the model artifact. load_model(...) restores the saved model. predict(...) runs inference with restored parameters. Compare original and loaded predictions to verify saving/loading.

Remember

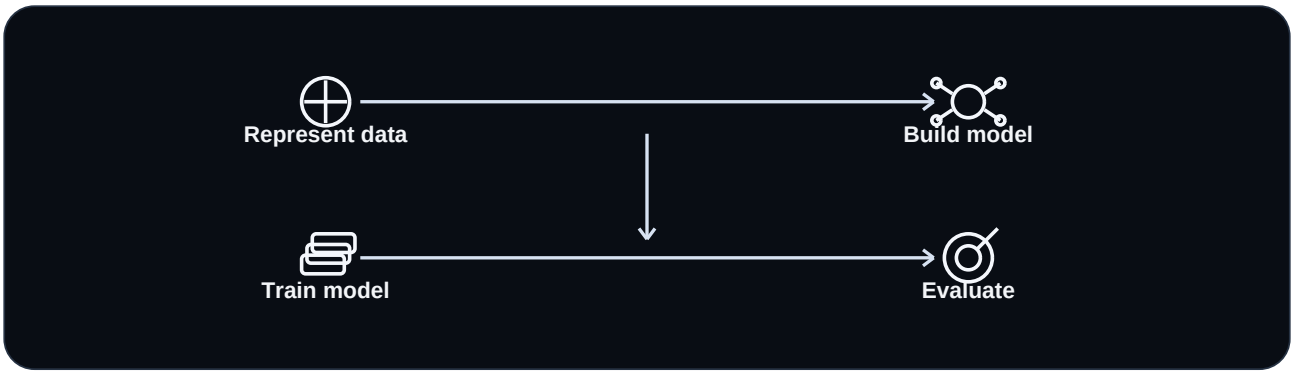
- model.save(...) writes the model artifact.

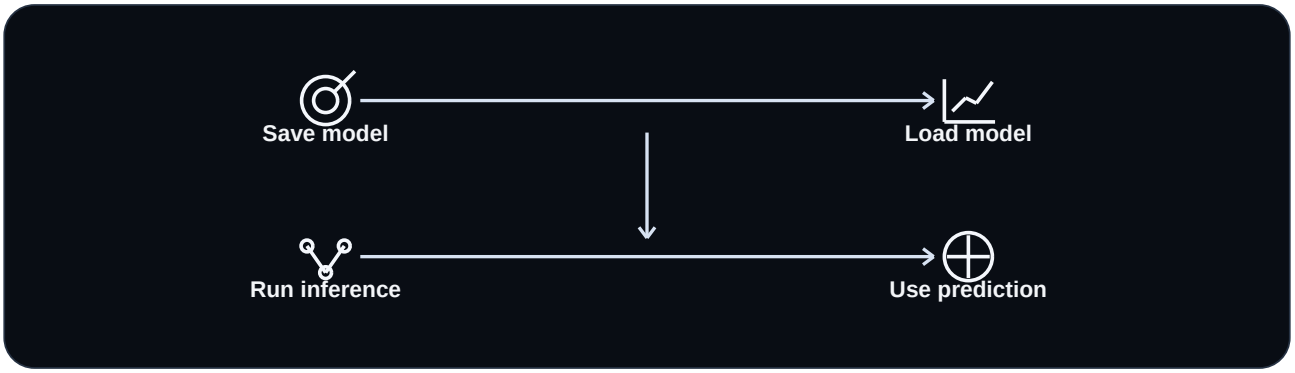
- load_model(...) restores the saved model.

- predict(...) runs inference with restored parameters.

- Compare original and loaded predictions to verify saving/loading.

The complete learning path





The first lessons built the mathematical pieces. The later lessons connected them into a basic machine-learning workflow: represent data, define a model, train its parameters, evaluate performance, and save/reuse the trained model.

REFERENCE | Shapes and vocabulary

Term	Meaning	Example
Scalar	One value	7.0; shape ()
Vector	One axis	[1,2,3]; shape (3,)
Matrix	Two axes	[[1],[2],[3]]; shape (3,1)
Batch	Examples grouped together	4 one-feature examples; shape (4,1)
dtype	Element data type	tf.float32, tf.int32
Rank	Number of dimensions	shape (4,1) has rank 2

Keras methods

Method	Purpose
Dense(units)	Creates a fully connected layer with the requested number of output units.
compile(optimizer, loss)	Configures how the model is trained.
fit(x, y, epochs)	Trains the model by updating its parameters.
evaluate(x, y)	Calculates loss and metrics; does not train.
predict(x)	Runs inference and returns outputs.
save(path)	Saves a model artifact.
load_model(path)	Restores a saved Keras model.

Troubleshooting

ModuleNotFoundError: No module named tensorflow

The script may be using a Python interpreter without TensorFlow. Check where python and python --version. From src, run ..\venv\Scripts\python.exe script.py.

(venv) appears, but where python is wrong

The prompt label does not prove which interpreter is running. Use the full path to venv\Scripts\python.exe.

Dense input shape error

Four examples with one feature each should be [[1.0],[2.0],[3.0],[4.0]], shape (4,1), not [1.0,2.0,3.0,4.0], shape (4,).

oneDNN message

Informational CPU optimization message; not a failure.

GPU unavailable on native Windows

Expected with the standard modern TensorFlow package on native Windows. CPU execution is fine for these exercises.

YOUR LEARNING LOG | Reuse for every new day

Use this page for Day 13, Day 14 and every new concept. Keep one day focused on one concept and one small runnable experiment.

DAY ____ | CONCEPT: _____

Date learned: _____

Goal: What should I understand or be able to do?

_____

Core idea in my own words:

_____
_____

Diagram: Sketch the flow of data or operations.

Formula / rule: _____

Code file: _____ Git commit: _____

What I ran:

_____
_____

Output and meaning:

_____
_____

What confused me / what to revisit:

_____
_____

One-sentence summary:

_____

GIT WORKFLOW

Save the lesson in src/, run it with the project's interpreter, then commit the day's file:
git add src/13_new_topic.py
git commit -m "learn [topic]"
git push